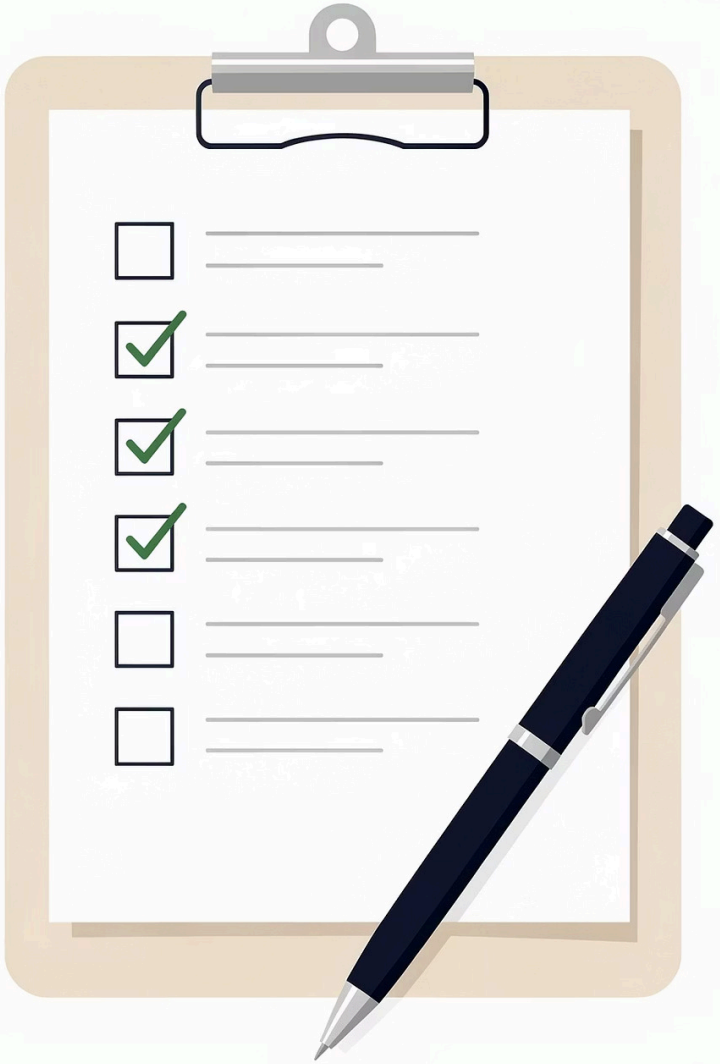


Verificación y Validación con Herramientas

UNIDAD 3 · SEMANA 2

Testing práctico, contratos, cobertura de código y metodologías de laboratorio para el desarrollo de software de alta confiabilidad.



Pruebas Unitarias y el Patrón AAA

La escritura de pruebas unitarias en el laboratorio profesional exige la validación sistemática de condiciones de éxito bajo estrés lógico y el control estricto de casos de borde. La estructura fundamental sigue el patrón **AAA**:

1. **Arrange**
   Configuración del escenario de prueba
2. **Act**
   Ejecución de la lógica bajo prueba
3. **Assert**
   Verificación de resultados mediante aseveraciones

Aseveraciones y Casos de Borde

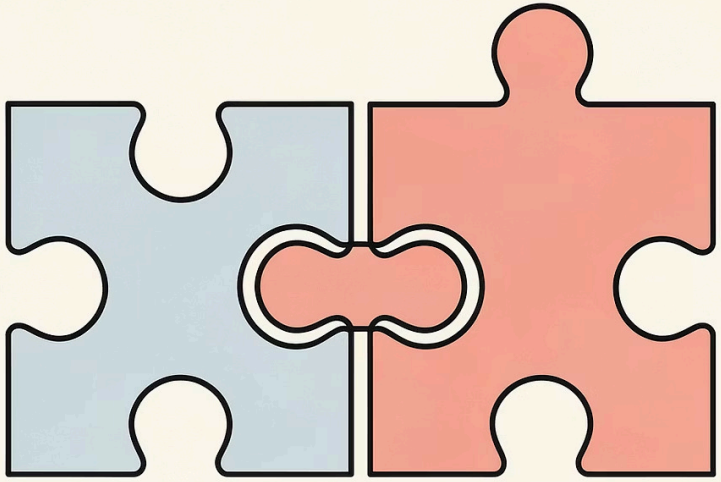
Tipos de Aseveraciones

- **Igualdad y estado:** Correspondencia exacta entre tipos primitivos u objetos complejos
- **Identidad y referencia:** Verifican si dos variables apuntan al mismo espacio de memoria
- **Colecciones:** Inspeccionan elementos, ordenamiento y longitud de arreglos o listas

Inyección de Errores Controlados

El software de alta confiabilidad debe validar los límites extremos de sus variables: valores nulos, cadenas vacías, desbordamientos numéricos y entradas fuera de rango.

Las pruebas deben forzar estos escenarios para verificar que los mecanismos de manejo de excepciones (`try/catch`) capturen la anomalía y respondan con una excepción controlada, evitando la caída abrupta de la aplicación.



Dobles de Prueba: Mocks y Stubs

En ecosistemas basados en microservicios, las pruebas unitarias deben garantizar el **aislamiento atómico absoluto** del componente bajo examen, sin llamadas reales a bases de datos o APIs externas.

Stubs

Objetos estáticos que reemplazan dependencias reales y devuelven respuestas preconfiguradas. Su propósito es **simular estados de infraestructura** y proveer entradas indirectas al componente.

Mocks

Dobles dinámicos y avanzados que **auditan el comportamiento** del componente: qué métodos invocó, con qué parámetros, en qué orden y cuántas veces. La aseveración final interroga al Mock para validar el protocolo de interacción.

Cobertura de Código: Statement vs. Branch

La cobertura de código es un indicador auditable que mide la exhaustividad con la que la suite de pruebas ha inspeccionado el código fuente de producción.

Statement Coverage (Cobertura de Sentencias)

Calcula la proporción de líneas ejecutables recorridas al menos una vez. Alcanzar más del **80%** es un estándar industrial deseable, pero no garantiza la ausencia de bugs ante casos de borde no contemplados.

Branch Coverage (Cobertura de Ramas)

Criterio más riguroso: analiza todos los caminos condicionales posibles (`if`, `else`, `switch`). Exige al menos un caso que fuerce la condición a **verdadera** y otro a **falsa**, certificando que todas las rutas lógicas han sido validadas y reduciendo código muerto en producción.

Pruebas de Contrato en Entornos Distribuidos

El **Contract Testing** valida de forma aislada las interfaces de comunicación entre componentes independientes, asegurando compatibilidad sin costos de infraestructura.

Consumer (Consumidor)

Componente que inicia una petición de red y requiere los datos estructurados provistos por otro servicio para ejecutar su propia lógica de negocio.

Provider (Proveedor)

Servicio o API que expone un endpoint público y se compromete a procesar peticiones y entregar respuestas consistentes bajo reglas técnicas predefinidas.

Contrato Técnico

Archivo de especificación estricto (JSON u OpenAPI) que define nombres de campos, tipos de datos, headers y códigos HTTP válidos. Si el backend modifica un campo esperado, la prueba falla de inmediato en el entorno local.



Regresión Automatizada y Smoke Tests

Pruebas de Regresión

Cada cambio en el código introduce el riesgo de alterar componentes que ya funcionaban correctamente. Las pruebas de regresión consisten en la **reejecución sistemática y automatizada** de la suite completa ante cualquier modificación, actuando como red de seguridad técnica.

Smoke Test: 3 Estados Críticos

1. **Liveness:** El proceso del servidor inició y se mantiene activo
2. **Readiness:** Los puertos y sockets responden adecuadamente
3. **Healthcheck:** El endpoint base devuelve HTTP 200 OK con conexiones internas estables

TDD: Desarrollo Guiado por Pruebas

El TDD invierte el flujo tradicional: los criterios de aceptación se diseñan **antes** de escribir la solución funcional. Su ciclo iterativo tiene tres fases inmutables:

🔴 Fase Roja (RED)

Escribir una prueba unitaria para una funcionalidad aún no implementada. Debe fallar obligatoriamente, certificando que el test es válido y libre de falsos positivos.

🟢 Fase Verde (GREEN)

Escribir el código mínimo y necesario para que la prueba pase a estado exitoso. No se prioriza la optimización, sino la efectividad de la solución lógica.

🔵 Refactorización (REFACTOR)

Con el test en verde como red de seguridad, limpiar la sintaxis, eliminar duplicaciones, optimizar algoritmos y mejorar el diseño arquitectónico del componente.



BDD: Desarrollo Guiado por Comportamiento

El BDD evoluciona el TDD desplazando el foco hacia la **lógica de negocio y el comportamiento esperado** desde la perspectiva del usuario. Los escenarios se describen con la sintaxis semántica **Gherkin**:

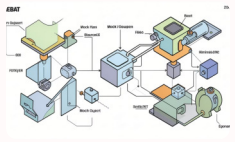
1	2	3
Dado (Given) Establece el contexto inicial y las precondiciones del sistema. *Ej: "Dado que el usuario tiene sesión activa y un saldo de $1,000"*	**Cuando (When)** Describe la acción concreta o transacción ejecutada por el actor. *Ej: "Cuando intenta transferir $500 al destinatario seleccionado"*	**Entonces (Then)** Especifica la consecuencia lógica observable que debe validar la aseveración. *Ej: "Entonces el sistema descuenta el dinero, emite un comprobante y actualiza el saldo a $500"*

Resumen: Pilares del Testing Profesional



Pruebas Unitarias

Patrón AAA, aseveraciones avanzadas e inyección de errores controlados para validar casos de borde.



Mocks y Stubs

Aislamiento atómico del componente: Stubs simulan estado, Mocks auditan comportamiento e interacciones.



Cobertura y Contratos

Statement y Branch Coverage como métricas auditables; Contract Testing para entornos distribuidos.



TDD y BDD

Ciclo Rojo-Verde-Refactor para TDD; sintaxis Gherkin (Dado/Cuando/Entonces) para BDD orientado al negocio.

Dominar estas prácticas garantiza software robusto, mantenible y libre de regresiones en entornos de producción.